in python 3.6, send json text and a binary file over a socket

In Python 3.6, you must serialize data into bytes before sending it over a socket. Because TCP is a stream-oriented protocol with no built-in message boundaries, you should implement a simple **prefix-length protocol** to ensure the receiver knows exactly how many bytes to read for each part. Stack Overflow +4

Sending JSON and Binary Data

The most efficient way to send both is to send the JSON metadata (which describes the file) first, followed by the raw binary file data. Stack Overflow +2

1. **Serialize JSON**: Convert your dictionary to a UTF-8 encoded byte string using json.dumps().
2. **Read Binary File**: Open the file in binary mode (`'rb'`) to read its raw bytes.
3. **Prefix with Length**: For both parts, send an integer representing the size of the data first so the receiver knows when to stop reading. Use the [struct module](#) to pack this integer into a fixed-size (e.g., 4-byte) header. Stack Overflow +5

Example Sender (Client)

python

```python
import socket
import json
import struct

def send_data(sock, json_meta, file_path):
```

Ask anything

```
    sock.sendall(struct.pack('!I', json_len))
    sock.sendall(json_bytes)

    # 4. Send File: [4-byte length][file_data]
    sock.sendall(struct.pack('!I', file_len))
    sock.sendall(file_bytes)
```

Use code with caution.

Receiving the Data

The receiver must perform the inverse: read the 4-byte length header first, then call `recv()` until that exact number of bytes is retrieved. Real Python Tutorials +1

Example Receiver (Server)

python

```python
def recv_all(sock, n):
    """Helper to receive exactly n bytes."""
    data = bytearray()
    while len(data) < n:
        packet = sock.recv(n - len(data))
        if not packet:
            return None
        data.extend(packet)
    return data

# Usage on accepted connection 'conn'
# Read JSON length and data
raw_json_len = recv_all(conn, 4)
json_len = struct.unpack('!I', raw_json_len)[0]
```

AI Mode All Images Videos News More

Use code with caution.

Alternative: Base64 Encoding

If you prefer to send everything in a single JSON object, you can encode the binary file as a Base64 string. However, this increases the data size by approximately 33% and consumes more memory, which may cause issues for very large files. Stack Overflow +4

Do you need to handle **large files** that might exceed available memory, or will these transfers be relatively small?

AI responses may include mistakes. Learn more

19 sites

Cannot send large binary files over Python socket

14 Apr 2022 — The client creates a dictionary containing metadata about the file to be sent. The binary file is base64 encoded and is...

Stack Overflow

File Transfer via Sockets in Python

12 Sept 2022 — what is going on guys welcome back in today's video we're going to learn how to transfer files via...

YouTube · NeuralNine

Implementing a simple TCP server using Python's socket. - Alex

6 Oct 2025 — You can send any binary data (files, images, etc.), but both sides must agree on the format. 2. Amount of Data. Each recv...

Medium

Show all

Sender: Memory-Efficient Transfer

This method sends the JSON metadata first, then streams the file content directly from the disk.

python

```python
import socket, json, struct, os

def send_large_file(sock, json_meta, file_path):
    # 1. Send JSON metadata (still small)
    json_bytes = json.dumps(json_meta).encode('utf-8')
    sock.sendall(struct.pack('!I', len(json_bytes)))
    sock.sendall(json_bytes)

    # 2. Send File Size so receiver knows when to stop
    file_size = os.path.getsize(file_path)
    sock.sendall(struct.pack('!Q', file_size)) # 'Q' for 8-byte unsig

    # 3. Stream File in Chunks
    with open(file_path, 'rb') as f:
        while True:
            chunk = f.read(8192) # Read 8KB at a time
            if not chunk:
                break
            sock.sendall(chunk)
```

Use code with caution.

Receiver: Writing to Disk in Chunks

The receiver reads exactly the number of bytes specified by the file size, writing each chunk to disk immediately to keep memory usage

```
    file_size = struct.unpack('!Q', sock.recv(8))[0]

    # 3. Receive and Save File in Chunks
    remaining = file_size
    with open(save_path, 'wb') as f:
        while remaining > 0:
            # Read only what's left or 8KB, whichever is smaller
            chunk_size = min(remaining, 8192)
            chunk = sock.recv(chunk_size)
            if not chunk:
                raise EOFError("Socket closed prematurely")
            f.write(chunk)
            remaining -= len(chunk)
    return json_meta
```

Use code with caution.

Key Techniques Used

- **Chunked I/O**: Only 8 KB of the file is in RAM at any moment, allowing you to transfer multi-gigabyte files on systems with limited memory.
- **Struct Padding**: Using `'!Q'` (8 bytes) for the file size allows handling files larger than 4GB, which exceeds the limit of `'!I'` (4 bytes).
- **Sequential Streams**: Since TCP is a continuous stream, the order of operations (JSON size → JSON → File size → File bytes) ensures the data is reconstructed correctly. Stack Overflow +4

Would you like to see how to add **progress tracking** or **checksum validation** to ensure the large file arrived without corruption?

AI responses may include mistakes. Learn more

AI Mode All Images Videos News More

YouTube · Technical Rajni

10 Ways to Work with Large Files in Python - Dev Genius

1 Dec 2024 — Data Analysis: Server logs, transaction records, or sensor data often come in gigantic files. Web Scraping: Processing...

Dev Genius

python sockets send file function explanation - Stack Overflow

10 Jan 2021 — 1 Answer. ... The program sends those 20 bytes and then the file payload itself. The receiver would read the 20 octets, convert th...

Stack Overflow

Show all

yes

To handle large files reliably in Python 3.6, you should integrate **hashing** (for data integrity) and a **progress bar** (for visibility).

1. Checksum Validation (Data Integrity)

Before sending, calculate the file's hash (e.g., SHA-256) in chunks to avoid memory spikes. Include this hash in your JSON metadata so the receiver can verify it after the transfer. Stack Overflow +1

2. Progress Tracking

Use the tqdm library to display a real-time progress bar. This library allows you to manually update the progress based on the number of bytes sent or received. tqdm documentation +3

Implementation Example

```
    # 1. Calculate SHA-256 Checksum in chunks
    sha256 = hashlib.sha256()
    with open(file_path, 'rb') as f:
        while True:
            chunk = f.read(8192)
            if not chunk: break
            sha256.update(chunk)
    checksum = sha256.hexdigest()

    # 2. Prepare and send Metadata
    metadata = {"filename": os.path.basename(file_path), "checksum": 
    meta_bytes = json.dumps(metadata).encode('utf-8')
    sock.sendall(struct.pack('!I', len(meta_bytes)))
    sock.sendall(meta_bytes)

    # 3. Send File Size and Stream Content with Progress Bar
    sock.sendall(struct.pack('!Q', file_size))
    with open(file_path, 'rb') as f, tqdm(total=file_size, unit='B', 
        while True:
            chunk = f.read(8192)
            if not chunk: break
            sock.sendall(chunk)
            pbar.update(len(chunk))
```

Use code with caution.

Receiver (Server)

python

```
def receive_large_file(sock, save_dir):
    # 1. Get Metadata
    meta_len = struct.unpack('!I', sock.recv(4))[0]
```

```
            chunk = sock.recv(min(remaining, 8192))
            if not chunk: break
            f.write(chunk)
            sha256.update(chunk)
            pbar.update(len(chunk))
            remaining -= len(chunk)

    # 3. Verify Checksum
    if sha256.hexdigest() == metadata['checksum']:
        print("\nSuccess: Checksum matches!")
    else:
        print("\nError: Corruption detected!")
```

Use code with caution.

Key Considerations

- **Format Codes**: `!I` is a 4-byte integer (up to 4GB); `!Q` is an 8-byte integer (up to 16 Exabytes), which is safer for "large" files.
- **Hash Choice**: SHA-256 is highly secure; for faster performance with very large files, you might consider blake2b.
- **Buffer Size**: `8192` (8 KB) is a standard buffer size that balances performance and memory usage. Python documentation +3

Would you like to explore **SSL/TLS wrapping** for this socket to ensure the data is encrypted during transfer?

AI responses may include mistakes. Learn more

9 sites

AI Mode All Images Videos News More

Real Python

struct — Interpret bytes as packed binary data — Python 3.14 ...

9 Mar 2026 — Format Characters * Format characters have the following meaning; the conversion between C and Python values...

Python documentation

Show all